

KARSA v0.3.1

API & Arsitektur Pipeline

Dokumentasi Teknis untuk Kontributor

Versi	v0.3.1
Tanggal	15 Juni 2026
Pipeline	Lexer - Parser - Resolver - Analyzer - Compiler
Test Suite	Lexer: 61, Parser: 95, Compiler: 105, Pipeline: PASS

1. Arsitektur Pipeline

KARSA menggunakan arsitektur pipeline 5 tahap yang mengkonversi kode sumber berbahasa Indonesia menjadi JavaScript yang dapat dieksekusi di browser. Setiap tahap memiliki tanggung jawab yang jelas dan terpisah, memungkinkan pengujian, debugging, dan pengembangan independen pada setiap komponen. Data mengalir searah dari Lexer hingga Compiler, dengan Engine sebagai orchestrator yang mengkoordinasikan seluruh proses.

1.1 Diagram Pipeline

```
Kode Sumber (.ks) | v [1] Lexer (karsa-lexer.js) | Input: string, Output: Token[] + Error[] v [2] Parser (karsa-parser.js) | Input: Token[], Output: AST + Error[] v [3] Resolver (karsa-resolver.js) | Input: AST, Output: AST + SemanticSymbol[] + Error[] v [4] Analyzer (karsa-analyzer.js) | Input: AST, Output: AST + Error[] + Warning[] v [5] Compiler (karsa-compiler.js) | Input: AST, Output: string (JavaScript) v JavaScript (DOM API)
```

1.2 Strategi Error Handling

Pipeline menggunakan dua strategi error handling yang dapat dipilih melalui opsi pada Engine. Mode fail-fast (default) menghentikan pipeline pada tahap pertama yang menghasilkan error, sehingga pengguna hanya melihat error dari satu tahap. Mode recovery (options.recover = true) melanjutkan pipeline meskipun ada error, mengumpulkan semua error dari seluruh tahap untuk ditampilkan sekaligus. Mode recovery sangat berguna selama pengembangan karena memungkinkan perbaikan banyak error sekaligus tanpa perlu menjalankan ulang kompilasi berulang kali.

1.3 Format Data Antar Tahap

Tahap	Input	Output	Error Code
Lexer	string	{ tokens, errors, warnings }	E1xxx / W1xxx
Parser	Token[]	{ ast, errors }	E2xxx / W2xxx
Resolver	AST	{ ast, errors, warnings } + semantic	E3xxx / W3xxx
Analyzer	AST	{ ast, errors, warnings }	E4xxx / W4xxx
Compiler	AST	string (JavaScript)	E5xxx / W5xxx
Engine	string	{ success, js, errors, warnings, ast, stages }	E0xxx / E6xxx

2. API Reference: Lexer

Lexer adalah tahap pertama pipeline yang bertugas mengkonversi stream karakter mentah menjadi stream token. Lexer menangani indentasi 2-spasi yang ketat (Python-style), keyword multi-kata via TRIE (misalnya "jika tidak", "tidak sama dengan"), komentar, DocString, selector CSS, dan region mentah "langsung:". Lexer tidak pernah throw; semua error dikembalikan melalui array errors.

2.1 API

```
KarsaLexer.tokenize(source: string) // Returns: { tokens: Token[], errors: LexerError[], warnings: object[] }
```

2.2 Struktur Token

Setiap token memiliki properti: tipe (TT constant), nilai (string literal), baris (nomor baris 1-indexed), kolom (nomor kolom 1-indexed), dan docstring (opsional, dari DocString yang menempel). Token tipe meliputi keyword KARSA (TK_BUAT, TK_DATA, TK_JIKA, dll.), literal (TK_LITERAL_ANGKA, TK_LITERAL_TEKS, TK_BENAR, TK_SALAH), operator (TK_PLUS, TK_MINUS, TK_SAMA_DENGAN, dll.), simbol struktur (TK_INDENT, TK_DEDENT, TK_BARIS_BARU), dan selector (TK_ATRIBUT, TK_CLASS, TK_ID).

2.3 Error Codes (E1xxx / W1xxx)

Kode	Pesan	Saran
E1001	Indentasi ganjil (bukan kelipatan 2)	Gunakan 2, 4, 6, atau 8 spasi
E1002	Karakter TAB di indentasi	Ganti tab menjadi spasi
E1003	DEDENT tidak cocok dengan level manapun	Periksa konsistensi indentasi
E1004	String tidak ditutup	Tambahkan kutip penutup
E1005	Karakter tidak dikenali	Periksa karakter sesuai spesifikasi
E1006	Komentar blok tidak ditutup	Tambahkan]]
E1009	Selector CSS tidak valid	Pastikan format #id, .class, tag
W1001	DocString tidak menempel ke node	Tempelkan ke deklarasi berikutnya

3. API Reference: Parser

Parser mengkonversi stream token menjadi Abstract Syntax Tree (AST) menggunakan Pratt Parser untuk ekspresi dan recursive descent untuk statement. Parser menjamin setiap node AST memiliki properti type dan loc (source location), serta menggunakan ErrorNode sebagai pengganti null pada posisi anak untuk memastikan traversal tidak terhenti.

3.1 API

```
KarsaParser.parse(tokens: Token[]) // Returns: { ast: ProgramNode, errors: ParseError[]
}
```

3.2 AST Node Factory

Semua node AST dibuat melalui ast-factory.js yang menjamin setiap node memiliki type dan loc (TIDAK PERNAH null/undefined). Jika loc tidak disediakan, digunakan UNKNOWN_LOC (0:0-0:0). Fungsi ensureLoc() memastikan fallback ini selalu diterapkan. Factory juga menyediakan buatLoc(), locFromTokens(), dan gabungLoc() untuk memanipulasi source location.

3.3 Error Codes (E2xxx / W2xxx)

Parser memiliki 28 kode error (E2001-E2028) dan 4 kode warning (W2001-W2004). Yang paling umum: E2001 (token tidak sesuai), E2004 (blok aksi diharapkan), E2010 (keyword tidak dikenali). Lihat file karsa-error-codes.js untuk daftar lengkap dengan pesan dan saran perbaikan.

4. API Reference: Resolver

Resolver melakukan resolusi identifier, manajemen scope, dan pelacakan penggunaan simbol. Resolver menggunakan model SemanticSymbol yang melacak kind (data, tetap, ubah, turunan, fungsi, komponen, parameter), scope, read/write count, dan references. Resolver juga menangani alias properti Indonesia ke JavaScript (misalnya "teks" menjadi "innerText", "panjang" menjadi "length") dan self-reference "ketika" tanpa target.

4.1 API

```
const resolver = new KarsaResolver(); resolver.resolve(ast: ProgramNode) // Returns: {
ast, errors: object[], warnings: object[] } // Side effect: ast.semantic = { symbols,
globalScope }
```

4.2 SemanticSymbol Model

Properti	Tipe	Deskripsi
name	string	Nama identifier
kind	string	data tetap ubah turunan fungsi komponen parameter
scope	string	global blok komponen iterasi watcher
isReactive	boolean	true untuk data dan turunan
isWritable	boolean	false untuk tetap dan turunan
readCount	number	Jumlah kali simbol dibaca
writeCount	number	Jumlah kali simbol ditulis

4.3 Error Codes (E3xxx / W3xxx)

E3001 (identifikasi tidak dideklarasikan), E3002 (simbol duplikat dalam scope), E3003 (menulis ke variabel tetap), E3004 (komponen digunakan sebelum deklarasi). Warning: W3001 (variabel tidak digunakan), W3002 (shadowing).

5. API Reference: Analyzer

Analyzer melakukan validasi semantik meliputi: tipe, reaktivitas, kontrol alur, dan lifecycle. Analyzer menggunakan context stack untuk melacak posisi traversal (inComponent, inFunction, loopDepth, handlerDepth, inTurunanExpr) dan menerapkan aturan validasi berdasarkan konteks.

5.1 API

```
const analyzer = new KarsaAnalyzer(); analyzer.analyze(ast: ProgramNode) // Returns: {  
  ast, errors: object[], warnings: object[] }
```

5.2 Validasi yang Diterapkan

- E4001: Lifecycle hook hanya valid di dalam komponen
- E4004: Menulis ke data turunan (read-only) - menggunakan lookupSymbol() dan checkWriteToTurunan()
- E4005/E4006: Parameter duplikat dan urutan default parameter
- E4007: Mode tampilan tidak valid
- E6001/E6002/E6003: berhenti/lewati/kembalikan di luar konteks yang valid
- W4001: Type hint tidak cocok dengan nilai awal - mendukung Literal, ObjectLiteral, ArrayLiteral, BinaryExpression, UnaryExpression, CallExpression, MemberExpression

6. API Reference: Compiler

Compiler melakukan lowering AST menjadi Vanilla JavaScript DOM API. Compiler menggunakan Visitor Pattern (BaseVisitor) untuk traversal AST dan menghasilkan kode JavaScript yang self-contained dengan runtime helpers untuk reaktivitas berbasis Proxy. Output dibungkus dalam IIFE untuk isolasi scope.

6.1 API

```
const compiler = new KarsaCompiler(); compiler.compile(ast: ProgramNode) // Returns:  
string (JavaScript)
```

6.2 Runtime Helpers

Compiler menyisipkan runtime helpers secara otomatis di awal output. Helper yang tersedia: `__createReactive` (Proxy-based reactivity), `__createComputed` (computed/derived values), `__watch` (reactive watcher dengan cleanup), `__setState` (safe state mutation), `__createElement` (DOM creation), `__mount` (element mounting), `__cleanup` (memory leak prevention). Sistem reaktivitas menggunakan WeakMap untuk subscriber tracking dan Set untuk dependency tracking, dengan iterasi aman menggunakan salinan array.

6.3 Statement Compilation Map

Statement KARSA	Output JavaScript
data x = val	<code>const x = __createReactive(val)</code>
tetap x = val	<code>const x = val</code>
ubah x = val	<code>let x = val</code>
turunan x = expr	<code>const x = __createComputed(() => expr)</code>
buat tag#id	<code>document.createElement + appendChild</code>
tampilkan target	<code>__mount() / style.display = ""</code>
sembunyikan target	<code>style.display = "none"</code>
hapus target	<code>parentElement.removeChild(el)</code>
perbarui prop target	property mapping + <code>setAttribute</code>
ketika event:	<code>addEventListener(event, handler)</code>
saat x berubah:	<code>__watch(x, callback)</code>
jika cond:	<code>if (cond) { ... } else { ... }</code>
ulangi item dari src:	for/forEach loop
selama cond:	<code>while (cond) { ... }</code>
simpan val ke target	<code>__setState(target, val)</code>
tambahkan N ke target	<code>__setState(x, x.value + N)</code>
kurangi x dengan N	<code>__setState(x, x.value - N)</code>
sisipkan val ke arr	<code>arr.value.push(val)</code>
ambil dari url:	<code>fetch(url).then().catch()</code>
gunakan Komp:	<code>__komp_Komp(props) + appendChild</code>
arahkan ke url	<code>window.location.href = url</code>

7. API Reference: Engine

Engine adalah orchestrator yang menghubungkan seluruh tahap pipeline. Engine mendukung dua environment (Node.js dan Browser) dan menyediakan tiga method utama:

compile(), run(), dan init().

7.1 Karsa.compile(source, options)

```
Karsa.compile(source: string, options?: { recover?: boolean }) // Mode fail-fast
(default): // Returns: { success, js, errors, stage, warnings, ast } // // Mode recovery
(options.recover = true): // Returns: { success, js, errors, warnings, stages, ast } //
stages: { lexer, parser, resolver, analyzer, compiler } // each: { ran: boolean, errors:
number, reason?: string }
```

7.2 Karsa.run(source)

Menjalankan kode Karsa langsung di browser. Memanggil compile() lalu mengeksekusi JavaScript yang dihasilkan menggunakan document.createElement("script"). Jika kompilasi gagal, error ditampilkan di console.

7.3 Karsa.init()

Inisialisasi otomatis untuk tag script type="text/karsa". Memindai semua tag script dengan type text/karsa, mengambil konten (inline atau dari file .ks via src attribute), dan menjalankannya. Dipanggil otomatis saat DOMContentLoaded di browser.

8. Unified Error Code Registry

Semua kode error dan warning terdaftar di parser/karsa-error-codes.js dengan konvensi penomoran berikut: E1xxx/W1xxx (Lexer), E2xxx/W2xxx (Parser), E3xxx/W3xxx (Resolver), E4xxx/W4xxx (Analyzer), E5xxx/W5xxx (Compiler), E6xxx (Runtime/Engine), E0xxx (System). File ini juga menyediakan fungsi utilitas getSeverity(code) yang mengembalikan "error" atau "warning", getStage(code) yang mengembalikan nama tahap pipeline, createError(code, loc, overrides) untuk membuat objek error terformat, dan formatError(err) untuk format tampilan pengguna.

8.1 Error Code Summary

Tahap	Error	Warning	Total
Lexer	E1001-E1009 (9)	W1001 (1)	10
Parser	E2001-E2028 (28)	W2001-W2004 (4)	32
Resolver	E3001-E3004 (4)	W3001-W3002 (2)	6
Analyzer	E4001-E4010 (10)	W4001-W4004 (4)	14
Compiler	E5001-E5003 (3)	W5001-W5002 (2)	5
Runtime	E6001-E6004 (4)	-	4
System	E0000 (1)	-	1

Tahap	Error	Warning	Total
Total	59	13	72

9. AST Node Specification

Setiap node AST dibuat oleh ast-factory.js dan dijamin memiliki properti type (string) dan loc (SourceLocation: { start: {line, column}, end: {line, column} }). Jika loc tidak disediakan, digunakan UNKNOWN_LOC ({ start: {line:0, column:0}, end: {line:0, column:0} }). Tabel berikut mencantumkan semua tipe node beserta child keys yang digunakan oleh Visitor Pattern untuk traversal.

Node Type	Child Keys	Keterangan
Program	body	Root node
DataDeclaration	init	Reactive variable
TetapanDeclaration	init	Constant (const)
UbahDeclaration	init	Mutable (let)
TurunanDeclaration	init	Computed/derived
KomponenDeclaration	params, body	Component factory
FungsiDeclaration	params, body	Function declaration
BuatStatement	selector, properties, body, action	DOM creation
KetikaStatement	target, body, action	Event handler
SaatStatement	body	Reactive watcher
JikaStatement	condition, consequent, alternate	Conditional
UlangiStatement	source, body, rangeEnd	Loop
SelamaStatement	condition, body	While loop
SimpanStatement	value	State mutation
AmbilLuarStatement	url, options, branches	Fetch API
BinaryExpression	left, right	Binary operation
CallExpression	callee, arguments	Function call
ObjectLiteral	properties	Object literal
ArrayLiteral	elements	Array literal
Parameter	defaultValue	Function/component param